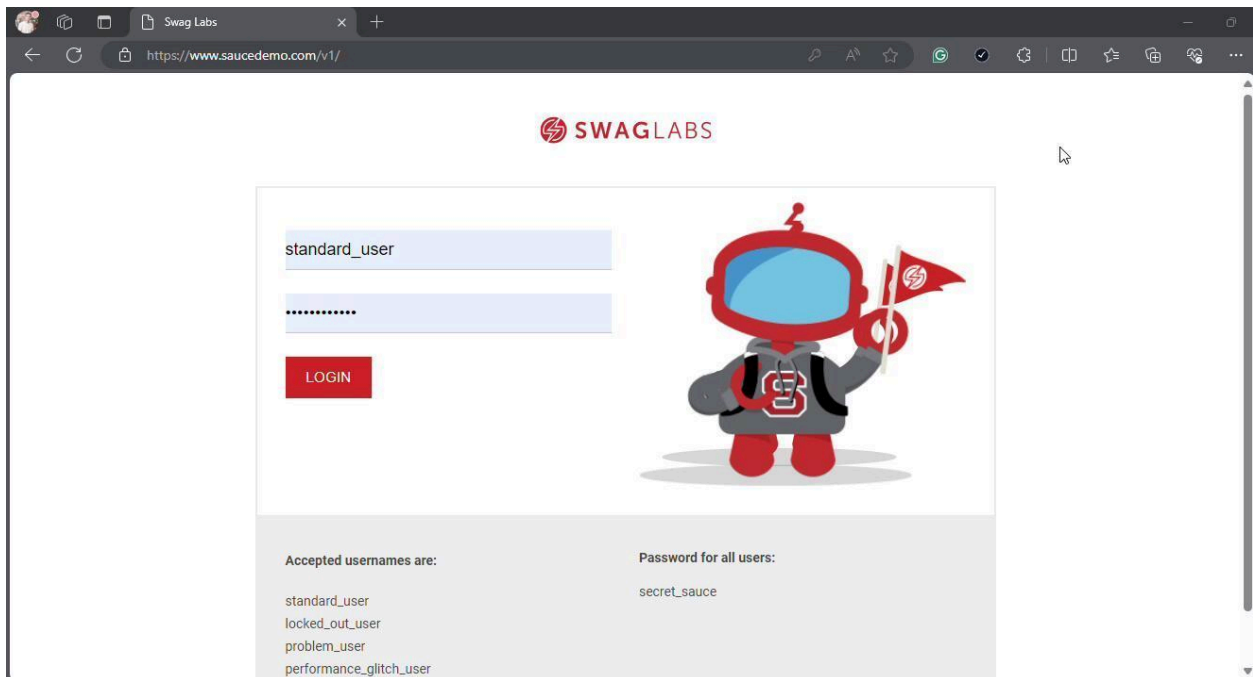


TUTORIAL – 11

PYTHON AUTOMATION TESTING

```
1 from selenium import webdriver
2 from selenium.webdriver import ActionChains
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.common.by import By
5 from selenium.webdriver.support import expected_conditions as EC
6 import time
7 driver = webdriver.Chrome()
8 url = 'https://oc.ebssw.kr/'
9 driver.get(url)
10 driver.maximize_window()
11 action = ActionChains(driver)
12 driver.find_element_by_css_selector('.nice-select').click()
13 driver.find_element_by_xpath('/html/body/div[1]/div[3]/div/div/div/div/div[2]/div[1]/div/ul/li[2]').click()
14 driver.find_element_by_xpath('/html/body/div[1]/div[3]/div/div/div/div/div[2]/div[2]/div/span').click()
15 driver.find_element_by_xpath('/html/body/div[1]/div[3]/div/div/div/div/div[2]/div[2]/div/ul/li[19]').click()
16 driver.find_element_by_xpath('/html/body/div[1]/div[3]/div/div/div/div/div[2]/div[3]/div/span').click()
17 driver.find_element_by_xpath('/html/body/div[1]/div[3]/div/div/div/div/div[2]/div[3]/div/ul/li[3]').click()
18 driver.find_element_by_xpath('/html/body/div[1]/div[3]/div/div/div/div/div[2]/div[4]/div/span').click()
19 driver.find_element_by_xpath('/html/body/div[1]/div[3]/div/div/div/div/div[2]/div[4]/div/ul/li[3]').click()
20 driver.find_element_by_xpath('/html/body/div[1]/div[3]/div/div/div/div/div[2]/div[5]/a').click()
21
22 WebDriverWait(driver, 20).until(EC.element_to_be_clickable(
23     (By.CSS_SELECTOR, ".my_menu>a")))
24 time.sleep(5)
```

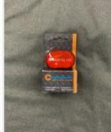




Sauce Labs Backpack
carry.allTheThings() with the sleek, streamlined Sly Pack that melds uncompromising style with unequaled laptop and tablet protection.

\$29.99

ADD TO CART



Sauce Labs Bike Light
A red light isn't the desired state in testing but it sure helps when riding your bike at night. Water-resistant with 3 lighting modes, 1 AAA battery included.

\$9.99

ADD TO CART



Sauce Labs Bolt T-Shirt
Get your testing superhero on with the Sauce Labs bolt T-shirt. From American Apparel, 100% ringspun combed cotton, heather gray with red bolt.

\$15.99

ADD TO CART



Sauce Labs Fleece Jacket
It's not every day that you come across a midweight quarter-zip fleece jacket capable of handling everything from a relaxing day outdoors to a busy day at the office.

\$49.99

ADD TO CART



Sauce Labs Onesie
Rib snap infant onesie for the junior automation engineer in development. Reinforced 3-snap bottom closure, two-needle hemmed sleeved and bottom won't unravel.

\$7.99

ADD TO CART



Test.allTheThings() T-Shirt (Red)
This classic Sauce Labs t-shirt is perfect to wear when cozying up to your keyboard to automate a few tests. Super-soft and comfy ringspun combed cotton.

\$15.99

ADD TO CART



Checkout: Overview

QTY	Description
1	Sauce Labs Backpack carry.allTheThings() with the sleek, streamlined Sly Pack that melds uncompromising style with unequaled laptop and tablet protection. \$29.99

Payment Information

SauceCard #31337

Shipping Information

Free Pony Express Delivery!

Price Total

Item total: \$29.99
Tax: \$2.40

Total: \$32.39

Cancel

Finish

[illegible]

Swag Labs

Products



Sauce Labs Backpack

carry.allTheThings() with the sleek, streamlined Sly Pack that melds uncompromising style with unequaled laptop and tablet protection.

\$29.99

Add to cart



Sauce Labs Bike Light

A red light isn't the desired state in testing but it sure helps when riding your bike at night. Water-resistant with 3 lighting modes, 1 AAA battery included.

\$9.99

Add to cart



Sauce Labs Bolt T-Shirt

Get your testing superhero on with the Sauce Labs bolt T-shirt. From American Apparel, 100% ringspun combed cotton, heather gray with red bolt.

\$15.99

Add to cart



Sauce Labs Fleece Jacket

It's not every day that you come across a midweight quarter-zip fleece jacket capable of handling everything from a relaxing day outdoors to a busy day at the office.

\$49.99

Add to cart

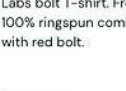


Sauce Labs Onesie

Official feature info for the Sauce Labs Onesie

\$39.99

Add to cart



Test.allTheThings() T-Shirt (Red)

Official feature info for the Sauce Labs Test.allTheThings() T-Shirt (Red)

\$19.99

Add to cart

1. Aim

To analyze automation testing in Python for website navigation, forms, multi-page workflows, links, and regression testing scenarios.

2. Theory

What is Automation Testing?

Automation testing means using a software tool to execute test steps automatically instead of manually performing every step.

For example, manually testing login:

1. Open Chrome.
2. Open website.
3. Type username.
4. Type password.
5. Click Login.
6. Check whether login is successful.

With Selenium, Python performs these actions automatically.

What is Selenium?

Selenium is an open-source tool used to automate web browsers.

What is Selenium WebDriver?

Selenium WebDriver allows Python programs to control a web browser.

The basic flow is:

Python Program → Selenium WebDriver → Chrome Browser → Web Application

What will we perform?

In this practical, students will automate:

- Website navigation
- Login form
- Links
- Product selection
- Cart
- Checkout
- Multi-page navigation
- Validation
- Regression testing

3. Application Used

We will use:

SauceDemo: <https://www.saucedemo.com/>

Login Credentials

Field	Value
Username	standard_user

Password	secret_sauce
----------	--------------

4. Software Required

Software	Requirement
Python	Python 3.x
IDE	PyCharm
Browser	Google Chrome
Automation Tool	Selenium WebDriver
Python Library	Selenium
Internet	Required

No Bugzilla, TestLink, VS Code or other testing software is required.

5. Step-by-Step Setup

Step 1 – Open PyCharm

Open **PyCharm**.

Select:

New Project → Create

Give the project name:

SoftwareTesting

Step 2 – Create Python File

In the project:

Right Click → New → Python File

Enter:

tutorial11.py

Step 3 – Install Selenium

At the bottom of PyCharm, click:

Terminal

Type:

pip install selenium

Press **Enter**.

Wait until installation completes.

To verify installation:

```
pip show selenium
```

If Selenium details appear, installation is successful.

6. First Selenium Program

Before performing the complete practical, students should understand the basic Selenium program.

```
from selenium import webdriver
```

```
driver = webdriver.Chrome()
```

```
driver.get("https://www.saucedemo.com/")
```

```
print(driver.title)
```

```
driver.quit()
```

Explanation Line by Line

Line 1

```
from selenium import webdriver
```

This imports Selenium's WebDriver module.

Purpose: Allows Python to control the browser.

Line 2

```
driver = webdriver.Chrome()
```

This creates a Chrome browser controlled by Selenium.

Meaning:

Open Google Chrome through Selenium.

Line 3

```
driver.get("https://www.saucedemo.com/")
```

`get()` opens the specified website.

Line 4

```
print(driver.title)
```

This prints the title of the webpage in the PyCharm console.

Line 5

```
driver.quit()
```

This closes the browser completely.

7. Complete Practical Procedure

Step 1 – Start Browser

```
driver = webdriver.Chrome()
```

Chrome opens automatically.

Step 2 – Maximize Browser

```
driver.maximize_window()
```

This maximizes the browser window.

Step 3 – Open Website

```
driver.get("https://www.saucedemo.com/")
```

Step 4 – Find Username Textbox

Inspect the username textbox.

The HTML element has:

```
id = user-name
```

Therefore:

```
driver.find_element(By.ID, "user-name")
```

Step 5 – Enter Username

```
driver.find_element(By.ID, "user-name").send_keys("standard_user")
```

Understand this:

```
find_element()
```

Finds the webpage element.

```
By.ID
```

Tells Selenium to find the element using its ID.

```
"user-name"
```

is the ID of the username textbox.

```
send_keys()
```

Types text into the textbox.

Step 6 – Enter Password

```
driver.find_element(By.ID, "password").send_keys("secret_sauce")
```

Step 7 – Click Login

```
driver.find_element(By.ID, "login-button").click()
```

`click()` performs a mouse-click action automatically.

Step 8 – Validate Login

After login, the application opens the Products page.

We can check the URL:

```
assert "inventory" in driver.current_url
```

Meaning

`assert` checks whether the expected condition is true.

If:

inventory

exists in the URL → **PASS**

Otherwise → **FAIL**

8. Add Product

After login:

```
driver.find_element(  
    By.ID,  
    "add-to-cart-sauce-labs-backpack"  
)
```

This clicks the **Add to Cart** button.

9. Open Cart

```
driver.find_element(  
    By.CLASS_NAME,  
    "shopping_cart_link"  
)
```

10. Checkout

```
driver.find_element(By.ID, "checkout").click()
```

11. Fill Checkout Form

```
driver.find_element(By.ID, "first-name").send_keys("Student")  
driver.find_element(By.ID, "last-name").send_keys("Testing")  
driver.find_element(By.ID, "postal-code").send_keys("380001")
```

12. Click Continue


```
driver.find_element(By.ID, "continue").click()
```

13. Validate Next Page

```
assert "checkout-step-two" in driver.current_url
```

14. Complete Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
# Step 1: Open Chrome
```

```
driver = webdriver.Chrome()
```

```
# Step 2: Maximize browser
```

```
driver.maximize_window()
```

```
# Step 3: Create explicit wait
```

```
wait = WebDriverWait(driver, 10)
```

```
try:
```

```
    # Step 4: Open website
```

```
    driver.get("https://www.saucedemo.com/")
```

```
    print("Website opened successfully")
```

```
    # Step 5: Enter username
```

```
    username = wait.until(
        EC.visibility_of_element_located(
            (By.ID, "user-name")
        )
    )
```

```
    username.send_keys("standard_user")
```

```
    # Step 6: Enter password
```

```
    password = driver.find_element(
        By.ID, "password"
    )
```

```
password.send_keys("secret_sauce")
```

```
# Step 7: Click Login  
driver.find_element(  
    By.ID, "login-button"  
).click()
```

```
# Step 8: Validate Products page  
wait.until(  
    EC.url_contains("inventory")  
)
```

```
assert "inventory" in driver.current_url
```

```
print("Login Test: PASS")
```

```
# Step 9: Add product  
wait.until(  
    EC.element_to_be_clickable(  
        (By.ID, "add-to-cart-sauce-labs-backpack")  
    )  
).click()
```

```
print("Product Added: PASS")
```

```
# Step 10: Open cart  
driver.find_element(  
    By.CLASS_NAME,  
    "shopping_cart_link"  
).click()
```

```
assert "cart" in driver.current_url
```

```
print("Cart Navigation: PASS")
```

```
# Step 11: Checkout  
driver.find_element(  
    By.ID, "checkout"  
).click()
```

```

# Step 12: Fill form
wait.until(
    EC.visibility_of_element_located(
        (By.ID, "first-name")
    )
).send_keys("Student")

driver.find_element(
    By.ID, "last-name"
).send_keys("Testing")

driver.find_element(
    By.ID, "postal-code"
).send_keys("380001")

# Step 13: Continue
driver.find_element(
    By.ID, "continue"
).click()

# Step 14: Validate next page
wait.until(
    EC.url_contains("checkout-step-two")
)

assert "checkout-step-two" in driver.current_url

print("Multi-Page Workflow: PASS")

finally:

    # Step 15: Close browser
    driver.quit()

print("Tutorial 11 Completed Successfully")

```

15. Important Code Concepts

Code	Meaning
------	---------

<code>webdriver.Chrome()</code>	Opens Chrome
<code>driver.get()</code>	Opens webpage
<code>find_element()</code>	Finds webpage element
<code>By.ID</code>	Finds element by ID
<code>By.CLASS_NAME</code>	Finds element by class
<code>send_keys()</code>	Enters text
<code>click()</code>	Clicks element
<code>driver.current_url</code>	Gets current URL
<code>assert</code>	Validates expected result
<code>WebDriverWait</code>	Waits for element/page condition
<code>driver.quit()</code>	Closes browser

16. How to Run

1. Save the file using **Ctrl + S**.
2. Right-click `tutorial11.py`.
3. Select **Run 'tutorial11'**.
4. Chrome will open.
5. Selenium performs the actions automatically.
6. Observe the browser.
7. Observe the PyCharm console.

Expected console:

Website opened successfully

Login Test: PASS

Product Added: PASS

Cart Navigation: PASS

Multi-Page Workflow: PASS

Tutorial 11 Completed Successfully

17. Regression Testing

Meaning

Regression testing means testing existing functionality again after changes to ensure that previously working features still work.

For this practical, students can execute:

Regression ID	Feature
RT-01	Login
RT-02	Product selection
RT-03	Cart
RT-04	Checkout
RT-05	Page navigation

18. Test Cases

TC ID	Test Case	Expected Result
TC-11-01	Open website	Website opens
TC-11-02	Valid login	Products page opens
TC-11-03	Add product	Product added
TC-11-04	Open cart	Cart opens
TC-11-05	Checkout	Checkout page opens
TC-11-06	Fill checkout	Data accepted
TC-11-07	Validate next page	Correct page opens

19. Screenshot Requirements

Students should capture:

Screenshot 1: PyCharm code

Screenshot 2: Login page

Screenshot 3: Products page

Screenshot 4: Cart

Screenshot 5: Checkout page

Screenshot 6: PyCharm console output

20. Observation

Sr. No.	Observation	Result
1	Website opened automatically	Successful

2	Login performed	Successful
3	Product added	Successful
4	Cart opened	Successful
5	Checkout performed	Successful

21. Questionnaire

Q1. Which Python library is commonly used for web automation testing?

- A) NumPy
- B) Pandas
- C) Selenium
- D) Matplotlib

Answer: C) Selenium

Q2. Selenium WebDriver is mainly used for?

Answer: Browser Automation

Q3. Which command installs Selenium?

Answer: `pip install selenium`

Q4. What is regression testing?

Answer: Testing existing functionality again after changes.

Q5. Which Selenium component is used for browser automation?

Answer: Selenium WebDriver

TUTORIAL – 12

AI-POWERED SOFTWARE TESTING

Your manual defines Practical 12 around **Generative AI for test-case generation, test-data creation, defect prediction, requirement analysis and automated Selenium script generation.**

1. Aim

To analyze AI-powered software testing tools and apply Generative AI techniques for test case generation, test data creation, defect prediction, requirement analysis, and automated Selenium test script generation for software quality assurance.

2. Theory

What is AI-Powered Software Testing?

AI-powered software testing uses Artificial Intelligence to assist testers in tasks such as:

- Requirement analysis
- Test case generation
- Test data generation
- Defect prediction
- Test script generation
- Test result analysis

Important

AI does **not** mean that students blindly copy AI-generated code.

The correct workflow is:

Requirement → AI Suggestion → Human Review → Correct Code → Execute → Verify

3. Practical Requirement

Consider:

A registered user should be able to log in using valid username and password.

Invalid credentials should be rejected and an appropriate error message should be displayed.

4. What Students Have to Do

Students will use Generative AI to generate:

1. Requirement analysis
2. Positive test cases
3. Negative test cases
4. Test data
5. Defect-prone areas
6. Selenium Python script
7. Regression test cases

Then students will:

Review → Correct → Copy to PyCharm → Run → Verify

5. AI Prompt

Students can use:

Analyze the following software requirement:

A registered user should be able to log in using a valid username and password. Invalid credentials should be rejected and an appropriate error message should be displayed.

Generate:

1. Positive test cases
2. Negative test cases
3. Test data
4. Expected results
5. Regression test cases
6. Possible defect-prone areas
7. Selenium Python automation code.

The Selenium Python code should be suitable for execution in PyCharm using Google Chrome.

6. Understand the AI Output

AI may generate something like:

TC ID	Scenario	Input	Expected Result
TC01	Valid Login	Valid username/password	Login successful
TC02	Wrong Password	Valid username/wrong password	Error
TC03	Wrong Username	Wrong username/valid password	Error
TC04	Blank Username	Blank	Validation
TC05	Blank Password	Blank	Validation

7. Test Data

For SauceDemo:

Valid

Username: standard_user

Password: secret_sauce

Invalid

Username: wrong_user

Password: wrong_password

8. AI-Based Defect Prediction

Ask AI:

For a login page, identify possible defect-prone areas related to username validation, password validation, authentication, error messages, session handling and logout. Give one possible defect example for each area.

Students should record the output and **verify it logically**.

9. AI-Generated Selenium Code

After AI generates the code, students must understand it before executing it.

Use this reviewed code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
```



```
from selenium.webdriver.support import expected_conditions as EC
```

```
driver = webdriver.Chrome()
```

```
driver.maximize_window()
```

```
wait = WebDriverWait(driver, 10)
```

```
try:
```

```
    driver.get("https://www.saucedemo.com/")
```

```
    username = wait.until(
        EC.visibility_of_element_located(
            By.ID, "user-name"
        )
    )
```

```
    username.send_keys("standard_user")
```

```
    password = driver.find_element(
        By.ID, "password"
    )
```

```
    password.send_keys("secret_sauce")
```

```
    login_button = driver.find_element(
        By.ID, "login-button"
    )
```

```
    login_button.click()
```

```
    wait.until(
        EC.url_contains("inventory")
    )
```

```
    assert "inventory" in driver.current_url
```

```
    print("AI-Assisted Login Test: PASS")
```

```
    print("Requirement Validation: PASS")
```

except AssertionError:

```
    print("AI-Assisted Login Test: FAIL")
```

finally:

```
    driver.quit()
```

10. Line-by-Line Explanation

Import WebDriver

```
from selenium import webdriver
```

Allows Python to control Chrome.

Import By

```
from selenium.webdriver.common.by import By
```

Used to identify webpage elements.

Import Wait

```
from selenium.webdriver.support.ui import WebDriverWait
```

Allows Selenium to wait until an element is ready.

Import Expected Conditions

```
from selenium.webdriver.support import expected_conditions as EC
```

Provides conditions such as:

- Element visible
- Element clickable
- URL contains text

Open Chrome

```
driver = webdriver.Chrome()
```

Started Chrome.

Maximize

```
driver.maximize_window()
```

Maximizes Chrome.

Wait

```
wait = WebDriverWait(driver, 10)
```

Selenium waits up to 10 seconds.

Open Website

```
driver.get("https://www.saucedemo.com/")
```

Opens SauceDemo.

Find Username

```
username = wait.until(  
    EC.visibility_of_element_located(  
        (By.ID, "user-name")  
    )  
)
```

Selenium waits until the username field becomes visible.

Enter Username

```
username.send_keys("standard_user")
```

Types the username.

Find Password

```
password = driver.find_element(  
    By.ID, "password"  
)
```

Finds password field.

Enter Password

```
password.send_keys("secret_sauce")
```

Types password.

Find Login Button

```
login_button = driver.find_element(  
    By.ID, "login-button"  
)
```

Finds Login button.

Click

```
login_button.click()
```

Clicks Login.

Validate URL

```
wait.until(  
    EC.url_contains("inventory")
```

)

Waits for the Products page.

Assertion

```
assert "inventory" in driver.current_url
```

Checks whether login succeeded.

Exception Handling

```
except AssertionError:
```

Runs if assertion fails.

Close Browser

```
finally:
```

```
    driver.quit()
```

Browser closes regardless of result.

11. Practical Procedure

1. Open PyCharm.
2. Create [tutorial12.py](#).
3. Install Selenium.
4. Open Generative AI.
5. Enter the requirement.
6. Generate test cases.
7. Generate test data.
8. Ask for defect prediction.
9. Ask AI to generate Selenium code.
10. Read and understand the generated code.
11. Correct any errors.
12. Copy the final code into PyCharm.
13. Run the program.
14. Check Chrome.
15. Check PyCharm console.
16. Compare actual result with expected result.
17. Record the result.
18. Take screenshots.

12. Test Case Execution

Step	Action	Expected	Actual	Status
------	--------	----------	--------	--------

1	Open website	Login page	Login page	Pass
2	Enter username	Accepted	Accepted	Pass
3	Enter password	Accepted	Accepted	Pass
4	Click Login	Products page	Products page	Pass
5	Validate URL	Inventory URL	Inventory URL	Pass

13. Regression Testing

Regression ID	Scenario	Expected
RT-01	Valid Login	Login successful
RT-02	Wrong Password	Error displayed
RT-03	Wrong Username	Error displayed
RT-04	Blank Username	Validation displayed

14. Screenshots

Students should capture:

1. AI prompt
2. AI-generated test cases
3. AI-generated test data
4. AI-generated code
5. Corrected code in PyCharm
6. Browser output
7. Console output

15. Observation

No.	Observation	Result
1	Requirement analyzed using AI	Completed
2	Test cases generated	Completed
3	Test data generated	Completed
4	Selenium code generated/reviewed	Completed

5	Code executed in PyCharm	Successful
---	--------------------------	------------

TUTORIAL – 13

SELENIUM ENVIRONMENT SETUP

Your supplied format specifically includes **Python, Selenium WebDriver, IDE, browser drivers and automation libraries**, followed by source code, test-case details, execution and observation.

1. Aim

To create and configure Python, Selenium WebDriver, PyCharm IDE, browser drivers and automation testing libraries for Selenium testing.

2. Theory

Before writing Selenium automation programs, we need an environment.

The environment consists of:

Python



PyCharm



Selenium Library



Selenium WebDriver



Google Chrome



Website

3. Important Terms

Python

Programming language used to write automation scripts.

PyCharm

IDE used to write and execute Python programs.

Selenium

Python library used for browser automation.

WebDriver

Allows Selenium to communicate with the browser.

Chrome

Browser on which the automation script executes.

4. Setup Procedure

Step 1 – Check Python

Open Windows Command Prompt or PyCharm Terminal:

```
python --version
```

Expected:

Python 3.x.x

Step 2 – Check pip

```
pip --version
```

Step 3 – Create PyCharm Project

Create:

SeleniumTesting

Step 4 – Create Python File

tutorial13.py

Step 5 – Install Selenium

In PyCharm Terminal:

```
pip install selenium
```

Step 6 – Verify Selenium

```
pip show selenium
```

If package information is displayed, Selenium is installed.

6. First Test Program

```
from selenium import webdriver
```

```
driver = webdriver.Chrome()
```

```
driver.maximize_window()
```

```
driver.get("https://www.selenium.dev/")
```

```
print("Browser launched successfully")
```

```
print("Page Title:", driver.title)
```

```
print("Current URL:", driver.current_url)
```

```
driver.quit()
```

7. Line-by-Line Explanation

Line 1

```
from selenium import webdriver
```

Imports Selenium WebDriver.

Line 2

```
driver = webdriver.Chrome()
```

Starts Google Chrome.

Line 3

```
driver.maximize_window()
```

Maximizes browser.

Line 4

```
driver.get("https://www.selenium.dev/")
```

Opens Selenium website.

Line 5

```
print("Browser launched successfully")
```

Displays a message in PyCharm.

Line 6

```
print("Page Title:", driver.title)
```

Reads webpage title.

Line 7

```
print("Current URL:", driver.current_url)
```

Displays current webpage URL.

Line 8

```
driver.quit()
```

Closes Chrome.

8. How to Execute

1. Save using **Ctrl + S**.
2. Right-click [tutorial13.py](#).
3. Select **Run 'tutorial13'**.
4. Chrome should open.

5. Selenium website should load.
6. Chrome should close.
7. Read console output.

Expected:

Browser launched successfully

Page Title: Selenium

Current URL: <https://www.selenium.dev/>

9. If Chrome Does Not Open

Students should check:

Check 1

Is Google Chrome installed?

Check 2

Is Python working?

`python --version`

Check 3

Is Selenium installed?

`pip show selenium`

Check 4

Try:

`python -m pip install --upgrade selenium`

Then run the program again.

10. Test Case

Field	Details
Test Case ID	TC-13-01
Requirement ID	REQ-13-01
Module	Selenium Environment
Feature	Browser Launch
Objective	Verify Selenium configuration
Test Type	System

Technique	Black Box
Priority	High
Severity	High
Preconditions	Python, PyCharm and Chrome installed
Test Data	Selenium website URL
Expected Result	Chrome opens successfully
Actual Result	_____
Status	_____

11. Test Execution

Step	Test Step	Expected	Actual	Status
1	Open PyCharm	IDE opens		
2	Check Python	Python available		
3	Install Selenium	Installation successful		
4	Run program	Chrome opens		
5	Open website	Website loads		
6	Print URL/title	Details displayed		

12. Screenshots

Students should capture:

1. Python version
2. PyCharm project
3. `pip install selenium`
4. `pip show selenium`
5. Source code
6. Chrome browser
7. Console output

13. Observation

No.	Observation	Remarks
-----	-------------	---------

1	Python verified	Successful
2	Selenium installed	Successful
3	PyCharm configured	Successful
4	Chrome launched	Successful
5	Website opened	Successful

TUTORIAL – 14

SELENIUM WEB AUTOMATION

Your manual specifies four major activities here: **browser navigation, form handling, web element interaction and validation testing.**

1. Aim

To create Selenium automation scripts in Python for browser navigation, form handling, web element interaction and validation testing.

2. Theory

Browser Navigation

Moving through a website automatically.

Examples:

`driver.get()`

`driver.back()`

`driver.forward()`

`driver.refresh()`

Form Handling

Entering information into:

- Username
- Password
- Textbox
- Search field
- Forms

Web Element Interaction

Interacting with:

- Buttons
- Textboxes
- Links
- Checkboxes
- Dropdowns

Validation Testing

Checking whether actual output matches expected output.

Example:

```
assert "inventory" in driver.current_url
```

3. Website

Use:

```
https://www.saucedemo.com/
```

Credentials:

Username: standard_user

Password: secret_sauce

4. Create File

Create:

```
tutorial14.py
```

5. Step-by-Step Procedure

Step 1 – Import Selenium

```
from selenium import webdriver
```

Step 2 – Import locator

```
from selenium.webdriver.common.by import By
```

Step 3 – Import wait

```
from selenium.webdriver.support.ui import WebDriverWait
```

Step 4 – Import condition

```
from selenium.webdriver.support import expected_conditions as EC
```

Step 5 – Start Chrome

```
driver = webdriver.Chrome()
```

Step 6 – Maximize

```
driver.maximize_window()
```

Step 7 – Create wait

```
wait = WebDriverWait(driver, 10)
```

Step 8 – Open website

```
driver.get("https://www.saucedemo.com/")
```

6. Login Form Handling

Username

```
driver.find_element(  
    By.ID,  
    "user-name"  
)send_keys("standard_user")
```

Password

```
driver.find_element(  
    By.ID,  
    "password"  
)send_keys("secret_sauce")
```

Login

```
driver.find_element(  
    By.ID,  
    "login-button"  
)click()
```

7. Validation

```
assert "inventory" in driver.current_url
```

If condition is true:

PASS

If false:

AssertionError

8. Product Interaction

```
driver.find_element(  
    By.ID,  
    "add-to-cart-sauce-labs-backpack"  
)click()
```

9. Cart Link

```
driver.find_element(  
    By.CLASS_NAME,
```

```
"shopping_cart_link"  
)click()
```

10. Complete Source Code

```
from selenium import webdriver  
from selenium.webdriver.common.by import By  
from selenium.webdriver.support.ui import WebDriverWait  
from selenium.webdriver.support import expected_conditions as EC
```

```
# Start Chrome  
driver = webdriver.Chrome()
```

```
# Maximize browser  
driver.maximize_window()
```

```
# Create wait object  
wait = WebDriverWait(driver, 10)
```

```
try:
```

```
# 1. Browser Navigation  
driver.get("https://www.saucedemo.com/")
```

```
print("Website opened")
```

```
# 2. Form Handling - Username  
username = wait.until(  
    EC.visibility_of_element_located(  
        (By.ID, "user-name")  
    )  
)
```

```
username.send_keys("standard_user")
```

```
# 3. Form Handling - Password  
password = driver.find_element(  
    By.ID, "password"  
)
```

```
password.send_keys("secret_sauce")
```

```
# 4. Click Login
driver.find_element(
    By.ID, "login-button"
).click()
```

```
# 5. Validation
wait.until(
    EC.url_contains("inventory")
)
```

```
assert "inventory" in driver.current_url
```

```
print("Login Validation: PASS")
```

```
# 6. Web Element Interaction
wait.until(
    EC.element_to_be_clickable(
        (By.ID, "add-to-cart-sauce-labs-backpack")
    )
).click()
```

```
print("Product Selection: PASS")
```

```
# 7. Navigate to Cart
driver.find_element(
    By.CLASS_NAME,
    "shopping_cart_link"
).click()
```

```
# 8. Validate Cart
assert "cart" in driver.current_url
```

```
print("Cart Validation: PASS")
```

```
finally:
```

```
# Close browser
driver.quit()
```

```
print("Tutorial 14 Completed Successfully")
```

11. Web Element Identification

Students should understand how to identify elements.

Element	Locator	Value
Username	ID	user-name
Password	ID	password
Login	ID	login-button
Add Product	ID	add-to-cart-sauce-labs-backpack
Cart	Class Name	shopping_cart_link

12. What is a Locator?

A locator tells Selenium:

Which element should I interact with?

Example:

By.ID, "user-name"

means:

Find the element whose ID is `user-name`.

13. Common Selenium Locators

Locator	Example
ID	By.ID
Name	By.NAME
Class Name	By.CLASS_NAME
Tag Name	By.TAG_NAME
Link Text	By.LINK_TEXT
CSS Selector	By.CSS_SELECTOR
XPath	By.XPATH

For beginners, **ID is usually the easiest locator** when a reliable ID is available.

14. How to Inspect Element

Students can identify locators using Chrome:

1. Open website.
2. Right-click textbox/button.
3. Select **Inspect**.
4. Developer Tools opens.
5. Find the HTML element.
6. Look for **id**, **name**, **class**, etc.
7. Use the locator in Python.

Example:

```
<input id="user-name">
```

Python:

```
By.ID, "user-name"
```

15. Test Case Execution

Step	Action	Expected Result	Status
1	Open website	Login page	Pass
2	Enter username	Text entered	Pass
3	Enter password	Text entered	Pass
4	Click Login	Products page	Pass
5	Add product	Product added	Pass
6	Click cart	Cart page	Pass
7	Validate URL	Correct URL	Pass

16. Validation Table

Validation	Expected	Actual	Status
Page Title	SauceDemo		
URL	inventory		
Login	Successful		
Product	Added		
Cart	Opened		

17. Screenshots

Take:

1. PyCharm code
2. Chrome Login
3. Chrome Products
4. Inspect Element
5. Cart
6. Console output

TUTORIAL – 15

AUTOMATED LOGIN TESTING

Your existing format specifically requires **valid login, invalid password, invalid username, empty credentials and logout**, followed by assertions and exception handling.

1. Aim

To create automated login functionality, authentication validation, assertions and exception handling scenarios using Selenium WebDriver with Python.

2. Theory

What is Authentication?

Authentication verifies the identity of a user.

Example:

Username + Password



Authentication



Access Granted / Access Denied

What is Login Testing?

Login testing checks whether:

- Valid user can login.
- Invalid user is rejected.
- Wrong password is rejected.
- Empty fields are validated.
- Logout works correctly.

3. What is Assertion?

An assertion compares expected and actual results.

Example:

```
assert "inventory" in driver.current_url
```

If the condition is true → test passes.

If false → test fails.

4. What is Exception Handling?

Sometimes Selenium cannot perform an action because:

- Element does not exist.
- Page takes too long.
- Element is not clickable.
- Page changes before interaction.

Python can handle these situations using:

try:

 # code

except:

 # error handling

finally:

 # cleanup

5. Exceptions Used

NoSuchElementException

Element cannot be found.

TimeoutException

Expected element/page does not appear within specified time.

ElementNotInteractableException

Element exists but cannot be interacted with.

StaleElementReferenceException

Previously found element is no longer attached to the current page DOM.

6. Test Scenarios

ID	Scenario	Expected
TC-01	Valid Login	Successful
TC-02	Invalid Password	Error
TC-03	Invalid Username	Error
TC-04	Empty Credentials	Validation
TC-05	Logout	Login page

7. Create File

Create:

tutorial15.py

8. Complete Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

from selenium.common.exceptions import (
    NoSuchElementException,
    TimeoutException,
    ElementNotInteractableException,
    StaleElementReferenceException
)

URL = "https://www.saucedemo.com/"

VALID_USERNAME = "standard_user"
VALID_PASSWORD = "secret_sauce"

def start_browser():

    driver = webdriver.Chrome()

    driver.maximize_window()

    return driver

def login(driver, username, password):

    driver.get(URL)

    wait = WebDriverWait(driver, 10)

    username_box = wait.until(
        EC.visibility_of_element_located(
            (By.ID, "user-name")
        )
    )
```

```
)
```

```
password_box = wait.until(  
    EC.visibility_of_element_located(  
        (By.ID, "password")  
    )  
)
```

```
login_button = wait.until(  
    EC.element_to_be_clickable(  
        (By.ID, "login-button")  
    )  
)
```

```
username_box.clear()  
username_box.send_keys(username)
```

```
password_box.clear()  
password_box.send_keys(password)
```

```
login_button.click()
```

```
# TC-01 VALID LOGIN
```

```
def test_valid_login():
```

```
    driver = start_browser()
```

```
    try:
```

```
        login(  
            driver,  
            VALID_USERNAME,  
            VALID_PASSWORD  
        )
```

```
        WebDriverWait(driver, 10).until(  
            EC.url_contains("inventory")  
        )
```

```
        assert "inventory" in driver.current_url

        print("TC-01 Valid Login: PASS")

    except AssertionError:

        print("TC-01 Valid Login: FAIL")

    except NoSuchElementException:

        print("NoSuchElementException handled")

    except TimeoutException:

        print("TimeoutException handled")

    except ElementNotInteractableException:

        print(
            "ElementNotInteractableException handled"
        )

    except StaleElementReferenceException:

        print(
            "StaleElementReferenceException handled"
        )

    finally:

        driver.quit()

# TC-02 INVALID PASSWORD
def test_invalid_password():

    driver = start_browser()

    try:
```

```

login(
    driver,
    VALID_USERNAME,
    "wrong_password"
)

error_message = WebDriverWait(
    driver, 10
).until(
    EC.visibility_of_element_located(
        (
            By.CSS_SELECTOR,
            "h3[data-test='error']"
        )
    )
)

assert (
    "Username and password do not match"
    in error_message.text
)

print("TC-02 Invalid Password: PASS")

except AssertionError:

    print("TC-02 Invalid Password: FAIL")

except TimeoutException:

    print("Error message not displayed")

finally:

    driver.quit()

# TC-03 INVALID USERNAME
def test_invalid_username():

```

```
driver = start_browser()
```

```
try:
```

```
    login(  
        driver,  
        "wrong_user",  
        VALID_PASSWORD  
    )
```

```
    error_message = WebDriverWait(  
        driver, 10  
    ).until(  
        EC.visibility_of_element_located(  
            (  
                By.CSS_SELECTOR,  
                "h3[data-test='error']"  
            )  
        )  
    )
```

```
    assert (  
        "Username and password do not match"  
        in error_message.text  
    )
```

```
    print("TC-03 Invalid Username: PASS")
```

```
except AssertionError:
```

```
    print("TC-03 Invalid Username: FAIL")
```

```
except TimeoutException:
```

```
    print("Error message not displayed")
```

```
finally:
```

```
    driver.quit()
```



```
# TC-04 EMPTY CREDENTIALS
```

```
def test_empty_credentials():
```

```
    driver = start_browser()
```

```
    try:
```

```
        driver.get(URL)
```

```
        login_button = WebDriverWait(  
            driver, 10
```

```
        ).until(  
            EC.element_to_be_clickable(  
                (By.ID, "login-button")  
            )  
        )
```

```
        login_button.click()
```

```
        error_message = WebDriverWait(  
            driver, 10
```

```
        ).until(  
            EC.visibility_of_element_located(  
                (  
                    By.CSS_SELECTOR,  
                    "h3[data-test='error']"  
                )  
            )  
        )
```

```
        assert (  
            "Username is required"  
            in error_message.text  
        )
```

```
        print("TC-04 Empty Credentials: PASS")
```

```
    except AssertionError:
```

```

        print("TC-04 Empty Credentials: FAIL")

except TimeoutException:

    print("Validation message not displayed")

finally:

    driver.quit()

# TC-05 LOGOUT
def test_logout():

    driver = start_browser()

    try:

        login(
            driver,
            VALID_USERNAME,
            VALID_PASSWORD
        )

        WebDriverWait(driver, 10).until(
            EC.url_contains("inventory")
        )

        menu_button = WebDriverWait(
            driver, 10
        ).until(
            EC.element_to_be_clickable(
                (By.ID, "react-burger-menu-btn")
            )
        )

        menu_button.click()

        logout_button = WebDriverWait(
            driver, 10

```

```
).until(  
    EC.element_to_be_clickable(  
        (By.ID, "logout_sidebar_link")  
    )  
)
```

```
logout_button.click()
```

```
WebDriverWait(driver, 10).until(  
    EC.visibility_of_element_located(  
        (By.ID, "login-button")  
    )  
)
```

```
print("TC-05 Logout: PASS")
```

```
except TimeoutException:
```

```
    print("Logout timed out")
```

```
except AssertionError:
```

```
    print("TC-05 Logout: FAIL")
```

```
finally:
```

```
    driver.quit()
```

```
# Run all tests
```

```
test_valid_login()
```

```
test_invalid_password()
```

```
test_invalid_username()
```

```
test_empty_credentials()
```

```
test_logout()
```

9. Detailed Code Explanation

Part 1 – Import WebDriver

```
from selenium import webdriver
```

Allows Python to start and control Chrome.

Part 2 – Import By

```
from selenium.webdriver.common.by import By
```

Used to identify webpage elements.

Part 3 – Import Wait

```
from selenium.webdriver.support.ui import WebDriverWait
```

Allows Selenium to wait for an element.

Part 4 – Expected Conditions

```
from selenium.webdriver.support import expected_conditions as EC
```

Provides conditions such as:

visibility_of_element_located

element_to_be_clickable

url_contains

10. Why We Use WebDriverWait

Suppose the browser takes 3 seconds to load a button.

If Selenium immediately searches for it, it might fail.

Instead:

```
WebDriverWait(driver, 10)
```

means:

Wait up to 10 seconds for the required condition.

This is better than unnecessarily using fixed delays such as:

```
time.sleep(10)
```

11. Understand the Login Function

```
def login(driver, username, password):
```

Creates a reusable function.

Instead of writing login code five times, we write it once and reuse it.

For example:

```
login(driver, "standard_user", "secret_sauce")
```

12. Valid Login

```
login(  
    driver,  
    VALID_USERNAME,  
    VALID_PASSWORD  
)
```

Sends correct credentials.

Then:

```
assert "inventory" in driver.current_url
```

checks successful authentication.

13. Invalid Password

```
login(  
    driver,  
    VALID_USERNAME,  
    "wrong_password"  
)
```

Correct username + wrong password.

Expected:

Error message

14. Invalid Username

```
login(  
    driver,  
    "wrong_user",  
    VALID_PASSWORD  
)
```

Wrong username + correct password.

Expected:

Error message

15. Empty Credentials

The student does not enter anything.

`driver.get(URL)`

opens login page.

Then:

`login_button.click()`

clicks Login.

Expected:

Username is required

16. Logout

First login.

Then:

`menu_button.click()`

opens menu.

Then:

`logout_button.click()`

logs out.

17. How to Execute Practical 15

1. Open PyCharm.
2. Open `tutorial15.py`.
3. Save the file.
4. Right-click the file.
5. Select **Run**.
6. Chrome opens.
7. TC-01 executes.
8. Browser closes.
9. TC-02 executes.
10. Browser closes.
11. TC-03 executes.
12. Browser closes.
13. TC-04 executes.
14. Browser closes.
15. TC-05 executes.
16. Browser closes.
17. Read PyCharm console.

Expected output will be similar to:

TC-01 Valid Login: PASS

TC-02 Invalid Password: PASS

TC-03 Invalid Username: PASS

TC-04 Empty Credentials: PASS

TC-05 Logout: PASS

18. Test Case Table

ID	Test Scenario	Input	Expected Result
TC-01	Valid Login	Valid username/password	Login successful
TC-02	Invalid Password	Valid username/wrong password	Error
TC-03	Invalid Username	Wrong username/valid password	Error
TC-04	Empty Credentials	Blank	Validation
TC-05	Logout	Valid login → Logout	Login page

19. Web Element Table

Element	Locator Type	Locator
Username	ID	user-name
Password	ID	password
Login Button	ID	login-button
Error Message	CSS Selector	h3[data-test='error']
Menu	ID	react-burger-menu-btn
Logout	ID	logout_sidebar_link

20. Assertion Validation

Assertion	Expected	Status
Login URL	Contains inventory	Pass
Invalid Password	Error displayed	Pass

Invalid Username	Error displayed	Pass
Empty Username	Username required	Pass
Logout	Login page displayed	Pass

21. Exception Handling

Exception	Meaning
NoSuchElementException	Selenium cannot find element
TimeoutException	Condition not achieved within wait time
ElementNotInteractableException	Element cannot be interacted with
StaleElementReferenceException	Previously located element is no longer valid

22. Screenshot Requirements

Students should take:

1. PyCharm source code
2. Valid Login
3. Invalid Password
4. Invalid Username
5. Empty Credentials
6. Logout
7. Console output
8. Exception-handling code

23. Observation

No.	Observation	Result
1	Valid authentication tested	Successful
2	Invalid password tested	Error displayed
3	Invalid username tested	Error displayed
4	Empty credentials tested	Validation displayed
5	Logout tested	Successful

24. Questionnaire

Q1. What is Authentication Testing?

Answer: Authentication testing verifies whether valid users are allowed to access an application and invalid users are rejected.

Q2. What is Selenium WebDriver?

Answer: Selenium WebDriver is an API that allows programs to control web browsers automatically.

Q3. What is Assertion?

Answer: An assertion compares the expected result with the actual result.

Q4. What is Exception Handling?

Answer: Exception handling manages runtime errors so that the program can respond properly instead of terminating unexpectedly.

Q5. Which exception occurs when an element cannot be found?

Answer: **NoSuchElementException**

STUDENT MASTER CHECKLIST – PRACTICAL 11 TO 15

Practical	Main Activity	Python File
11	Navigation + Forms + Multi-page + Regression	tutorial11.py
12	Generative AI + Test Cases + Test Data + Selenium	tutorial12.py
13	Python + PyCharm + Selenium Setup	tutorial13.py
14	Navigation + Forms + Elements + Validation	tutorial14.py
15	Login + Assertions + Exceptions + Logout	tutorial15.py

Common sequence students should follow**1. Open PyCharm****2. Create project****3. Create [.py](#) file****4. Install Selenium****5. Copy/type code****6. Save****7. Run**

8. Observe Chrome



9. Observe PyCharm console



10. Compare Expected vs Actual



11. Take screenshots



12. Fill Test Case / Observation tables



13. Complete Conclusion

IMPORTANT STUDENT EXECUTION CHECKLIST

Tutorial 11

- ☐ Create **tutorial11.py** in PyCharm
- ☐ Install Selenium
- ☐ Open Chrome using Selenium
- ☐ Perform login
- ☐ Navigate to products
- ☐ Add product
- ☐ Open cart
- ☐ Complete checkout form
- ☐ Validate multi-page workflow
- ☐ Execute regression scenario
- ☐ Capture output screenshots

Tutorial 12

- ☐ Analyze requirement using Generative AI
- ☐ Generate test cases
- ☐ Generate test data
- ☐ Perform defect prediction
- ☐ Generate/review Selenium code
- ☐ Execute final code in PyCharm
- ☐ Prepare regression test cases
- ☐ Capture AI + PyCharm + browser evidence

Tutorial 13

- ☐ Open PyCharm
- ☐ Configure Python interpreter
- ☐ Install Selenium
- ☐ Create Python file
- ☐ Execute Selenium program

- ☐ Launch Chrome
- ☐ Verify website
- ☐ Capture console output

Tutorial 14

- ☐ Open website
- ☐ Handle login form
- ☐ Enter username
- ☐ Enter password
- ☐ Click login
- ☐ Validate URL
- ☐ Select product
- ☐ Open cart
- ☐ Validate result
- ☐ Capture screenshots

Tutorial 15

- ☐ Valid Login
- ☐ Invalid Password
- ☐ Invalid Username
- ☐ Empty Credentials
- ☐ Assertions
- ☐ Error Message Validation
- ☐ Exception Handling
- ☐ Logout
- ☐ Execute all test cases
- ☐ Capture PyCharm console output

COMMON SOFTWARE REQUIREMENT FOR ALL PRACTICALS 11–15

Software/Tool	Requirement
IDE	PyCharm
Programming Language	Python
Automation Library	Selenium
Browser	Google Chrome
Testing Tool	Selenium WebDriver
Additional Testing Software	Not Required
Bugzilla	Not Required

TestLink	Not Required
VS Code	Not Required
Execution Method	Run <code>.py</code> file from PyCharm

Important: Students should use the faculty GitHub repositories shared by the faculty **only as a reference for understanding the procedure and implementation**. Do not directly copy-paste code, reports, screenshots, or outputs. Students must execute the practical themselves and submit their own screenshots and results.